Szybka transformata Fouriera (FFT) na rdzeniu RISC-V z rozszerzeniem o instrukcje arytmetyki zespolonej

5 listopada 2026

Spis treści

1 Dyskretna transformata Fouriera (DFT) 2
- 1.1 Macierz DFT . . . 2
- 1.2 Od DFT do FFT . . . 2
- 1.3 Motylek DIF . . . 6

2 Platforma sprzętowa: rdzeń CV32E40X z koprocesorem 6
- 2.1 Rdzeń CV32E40X i interfejs rozszerzeń . . . 6
- 2.2 Przebieg wykonania instrukcji . . . 7
- 2.3 Parametry rdzenia . . . 7
- 2.4 Używane i nieużywane interfejsy XIF . . . 8
- 2.5 Kodowanie instrukcji . . . 8
- 2.6 Odpowiedź koprocesora na zapytanie issue . . . 9
- 2.7 Wywołanie instrukcji z programu . . . 10

3 Reprezentacja danych: format Q15 10
- 3.1 Liczby stałoprzecinkowe Q15 . . . 10
- 3.2 Pakowanie liczby zespolonej w rejestr 32-bitowy . . . 11
- 3.3 Skalowanie – flaga h . . . 11

4 Rozszerzenie RISC-V: instrukcje zespolone 11
- 4.1 `CCVT` – konwersja próbek z ADC . . . 12
- 4.2 `CADD` – dodawanie z obrotem . . . 13
- 4.3 `CMLA` – zespolone mnożenie z akumulacją . . . 13
- 4.4 Realizacja motylków za pomocą instrukcji . . . 14
- 4.5 Przykład: FFT 4-punktowa bez pętli . . . 16

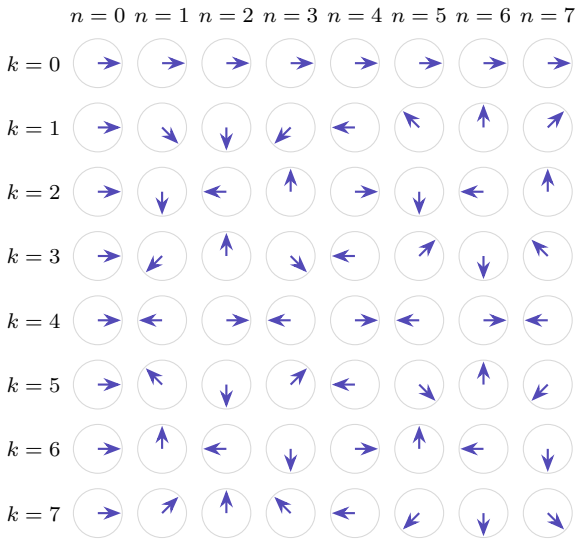
1 Dyskretna transformata Fouriera (DFT)

1.1 Macierz DFT

Jeżeli pominie się wartości próbek i narysuje same czynniki W_N^{kn} (wektory o długości 1), otrzymuje się macierz DFT. Obliczenie transformaty to pomnożenie wektora próbek przez tę macierz:

$$\begin{bmatrix} X[0] \\ X[1] \\ \vdots \\ X[N-1] \end{bmatrix} = \begin{bmatrix} W_N^0 & W_N^0 & \cdots & W_N^0 \\ W_N^0 & W_N^1 & \cdots & W_N^{N-1} \\ \vdots & \vdots & \ddots & \vdots \\ W_N^0 & W_N^{N-1} & \cdots & W_N^{(N-1)(N-1)} \end{bmatrix} \begin{bmatrix} x[0] \\ x[1] \\ \vdots \\ x[N-1] \end{bmatrix}.$$

Wiersz k zawiera wektory obracające się k razy szybciej niż w wierszu 1 (rys. 1).

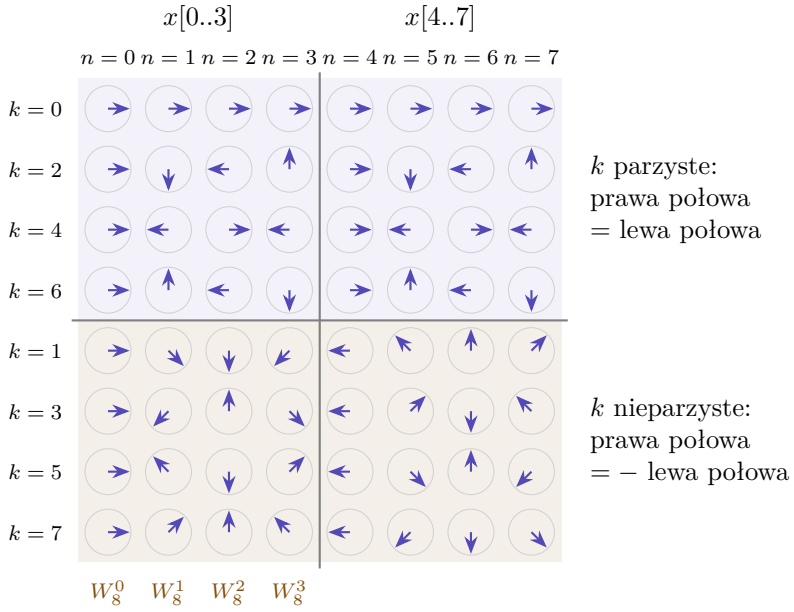


Rysunek 1: Macierz DFT dla $N = 8$: wektor w wierszu k i kolumnie n to W_8^{kn}. Wiersz $k = 0$ zawiera same jedynki (suma próbek), wiersz $k = 4$ – na przemian $+1$ i -1.

W macierzy powtarzają się te same wektory. Ze względu na własność $W_N^{N/2} = -1$ wektor w kolumnie $n + 4$ jest równy wektorowi w kolumnie n obróconemu o $k \cdot 180°$. Dla parzystych k oba wektory są identyczne, dla nieparzystych – przeciwne. Na tej obserwacji opiera się FFT.

1.2 Od DFT do FFT

Przestawmy wiersze macierzy tak, aby najpierw wystąpiły parzyste k, a potem nieparzyste, i podzielmy kolumny na dwie połowy (rys. 2).



Rysunek 2: Macierz DFT z wierszami ułożonymi w kolejności k parzyste, k nieparzyste. Lewa górna ćwiartka to macierz DFT dla $N = 4$. Lewa dolna ćwiartka to ta sama macierz, w której kolumna n jest dodatkowo obrócona o W_8^n (wartości pod rysunkiem).

Z rysunku wynika, że w wierszach parzystych próbki $x[n]$ i $x[n+4]$ są mnożone przez ten sam wektor, a w wierszach nieparzystych – przez wektory przeciwne. Można je więc najpierw zsumować lub odjąć, a mnożenie wykonać tylko raz:

$$X[k] = \sum_{n=0}^{N/2-1} \Big(x[n] + (-1)^k \, x[n + \tfrac{N}{2}] \Big) W_N^{kn}.$$

Rozdzielając parzyste ($k = 2r$) i nieparzyste ($k = 2r+1$) indeksy oraz korzystając z równości $W_N^2 = W_{N/2}$:

$$X[2r] = \sum_{n=0}^{N/2-1} \underbrace{\big(x[n] + x[n + \tfrac{N}{2}]\big)}_{g[n]} \, W_{N/2}^{rn},$$

$$X[2r+1] = \sum_{n=0}^{N/2-1} \underbrace{\big(x[n] - x[n + \tfrac{N}{2}]\big) \, W_N^n}_{h[n]} \, W_{N/2}^{rn}.$$

Etap 1: podział DFT 8-punktowej na dwie DFT 4-punktowe

Oba wzory mają tę samą postać: suma po $n = 0, \ldots, N/2-1$ ciągu pomnożonego przez $W_{N/2}^{rn}$. Jest to definicja DFT o długości $N/2$. Oznacza to, że:

- współczynniki o parzystych indeksach $X[0], X[2], X[4], X[6]$ są DFT 4-punktową ciągu $g[n]$,
- współczynniki o nieparzystych indeksach $X[1], X[3], X[5], X[7]$ są DFT 4-punktową ciągu $h[n]$.

Dla $N = 8$ ciągi $g[n]$ i $h[n]$ mają po 4 elementy:

$$\begin{aligned} g[0] &= x[0] + x[4], & h[0] &= (x[0] - x[4]) \cdot W_8^0, \\ g[1] &= x[1] + x[5], & h[1] &= (x[1] - x[5]) \cdot W_8^1, \\ g[2] &= x[2] + x[6], & h[2] &= (x[2] - x[6]) \cdot W_8^2, \\ g[3] &= x[3] + x[7], & h[3] &= (x[3] - x[7]) \cdot W_8^3. \end{aligned}$$

Jest to pierwszy etap grafu z rys. 4: sumy trafiają do górnej połowy tablicy ($c[0..3] = g[0..3]$), różnice pomnożone przez W_8^n – do dolnej ($c[4..7] = h[0..3]$). Zamiast jednej DFT 8-punktowej pozostają do obliczenia dwie DFT 4-punktowe.

Etap 2: podział DFT 4-punktowej na dwie DFT 2-punktowe

Oznaczmy przez $G[r]$ DFT 4-punktową ciągu $g[n]$. Z etapu 1 wiadomo, że $G[r] = X[2r]$. Dla $N = 4$ czynnik obrotu wynosi $W_4 = e^{-j2\pi/4} = -j$, więc z definicji:

$$\begin{aligned} G[0] &= g[0] + g[1] + g[2] + g[3], \\ G[1] &= g[0] - j\,g[1] - g[2] + j\,g[3], \\ G[2] &= g[0] - g[1] + g[2] - g[3], \\ G[3] &= g[0] + j\,g[1] - g[2] - j\,g[3]. \end{aligned}$$

Grupując składniki z $g[0], g[2]$ oraz z $g[1], g[3]$, otrzymuje się:

$$\begin{aligned} G[0] &= (g[0] + g[2]) + (g[1] + g[3]), & G[1] &= (g[0] - g[2]) + (-j)(g[1] - g[3]), \\ G[2] &= (g[0] + g[2]) - (g[1] + g[3]), & G[3] &= (g[0] - g[2]) - (-j)(g[1] - g[3]). \end{aligned}$$

Jest to dokładnie ten sam podział co w etapie 1, tylko dla $N = 4$:

- pary odległe o $N/2 = 2$: $(g[0], g[2])$ oraz $(g[1], g[3])$,
- sumy $g[0] + g[2]$, $g[1] + g[3]$ prowadzą do $G[0], G[2]$,
- różnice mnożone są przez W_4^n: $W_4^0 = 1$ dla $n = 0$ oraz $W_4^1 = -j$ dla $n = 1$, i prowadzą do $G[1], G[3]$.

Czynnik W_4^1 jest równy $W_8^2 = -j$ (ponieważ $W_N^2 = W_{N/2}$), dlatego mnożenie w tym etapie nie wymaga mnożarki.

Ciąg $h[n]$ przetwarza się identycznie. Jego DFT $H[r] = X[2r + 1]$, więc sumy prowadzą do $H[0] = X[1]$ i $H[2] = X[5]$, a różnice – do $H[1] = X[3]$ i $H[3] = X[7]$.

W grafie z rys. 4 odpowiada to wektorom `d` i `e`:

$$\begin{aligned} d[0] &= c[0] + c[2], & d[2] &= c[0] - c[2], & e[2] &= d[2], \\ d[1] &= c[1] + c[3], & d[3] &= c[1] - c[3], & e[3] &= d[3] \cdot (-j), \end{aligned}$$

i analogicznie dla dolnej połowy ($c[4..7]$, $d[4..7]$, $e[4..7]$).

Etap 3: DFT 2-punktowa

Po etapie 2 pozostają cztery DFT 2-punktowe. Dla $N = 2$ czynnik obrotu wynosi $W_2 = e^{-j\pi} = -1$, więc DFT dwóch liczb $y[0], y[1]$ to:

$$Y[0] = y[0] + y[1], \qquad Y[1] = y[0] - y[1].$$

Jest to motylek bez mnożenia. Dla górnej połowy tablicy:

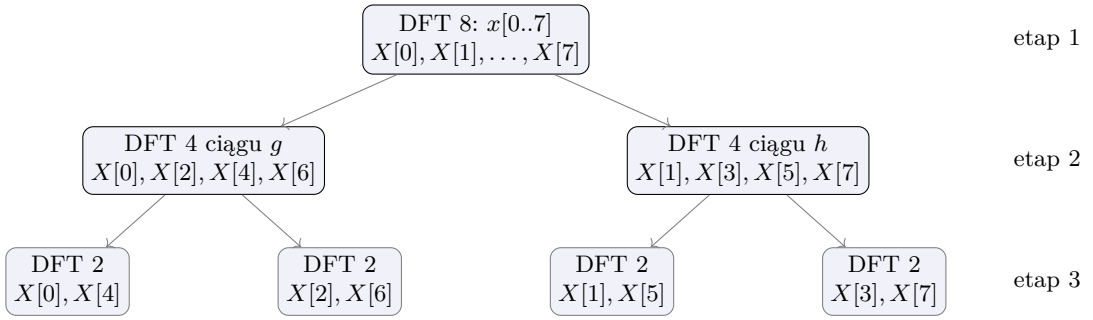
$$\begin{aligned} f[0] &= e[0] + e[1] = G[0] = X[0], & f[1] &= e[0] - e[1] = G[2] = X[4], \\ f[2] &= e[2] + e[3] = G[1] = X[2], & f[3] &= e[2] - e[3] = G[3] = X[6], \end{aligned}$$

a dla dolnej:

$$\begin{aligned} f[4] &= e[4] + e[5] = X[1], & f[5] &= e[4] - e[5] = X[5], \\ f[6] &= e[6] + e[7] = X[3], & f[7] &= e[6] - e[7] = X[7]. \end{aligned}$$

Podsumowanie etapów

Każdy etap dzieli każdą pozostałą DFT na dwie dwukrotnie krótsze (rys. 3). Po $\log_2 N$ etapach zostają DFT 2-punktowe, które są pojedynczymi motylkami.



Rysunek 3: Kolejne podziały DFT 8-punktowej. Każdy etap rozdziela indeksy parzyste (lewa gałąź) i nieparzyste (prawa gałąź).

etap	długość DFT	liczba DFT	odległość w parze	czynniki obrotu	motylki
1	8	1	4	$W_8^0, W_8^1, W_8^2, W_8^3$	4
2	4	2	2	$W_4^0 = 1,\ W_4^1 = -j$	4
3	2	4	1	$W_2^0 = 1$	4

Tabela 1: Etapy FFT dla $N = 8$.

W każdym etapie wykonywanych jest $N/2$ motylków, a etapów jest $\log_2 N$. Łączna liczba motylków wynosi więc $\frac{N}{2}\log_2 N$ – dla $N = 8$ jest to 12, wobec 64 mnożeń zespolonych przy obliczaniu DFT z definicji.

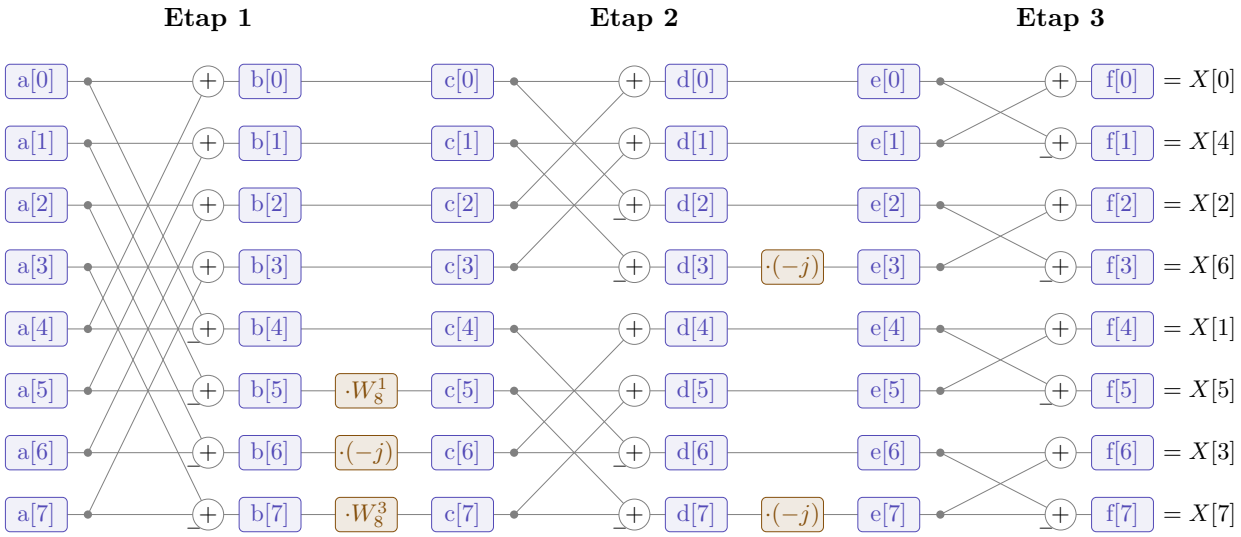
Kolejność wyników

Liście drzewa z rys. 3 czytane od lewej dają kolejność wyników w tablicy: $X[0]$, $X[4]$, $X[2]$, $X[6]$, $X[1]$, $X[5]$, $X[3]$, $X[7]$. Wynika to z tego, że każdy etap sprawdza jeden bit indeksu k, zaczynając od najmłodszego: etap 1 rozdziela k parzyste i nieparzyste (bit 0), etap 2 – bit 1, etap 3 – bit 2. Pozycja wyniku w tablicy jest więc indeksem k zapisanym w odwrotnej kolejności bitów (ang. *bit-reversed*).

pozycja w tablicy	binarnie	odwrócone bity	zawiera
0	000	000	$X[0]$
1	001	100	$X[4]$
2	010	010	$X[2]$
3	011	110	$X[6]$
4	100	001	$X[1]$
5	101	101	$X[5]$
6	110	011	$X[3]$
7	111	111	$X[7]$

Tabela 2: Kolejność wyników FFT dla $N = 8$.

Metoda nosi nazwę decymacji w częstotliwości (DIF, *decimation in frequency*), ponieważ w każdym etapie dzielony jest zbiór wyznaczanych częstotliwości k. Pełny graf przepływu dla $N = 8$ przedstawia rys. 4.

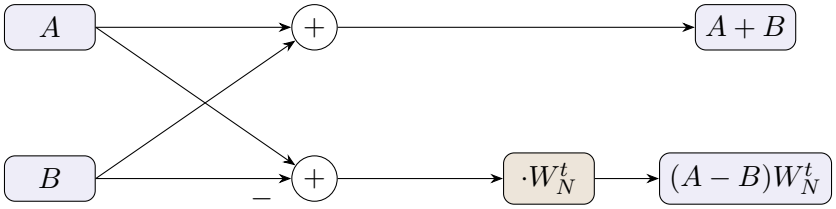


Rysunek 4: Graf przepływu FFT DIF dla $N = 8$. Kółko oznacza sumę wejść; wejście oznaczone „−” jest odejmowane. Wyjścia w kolejności bit-reversed.

1.3 Motylek DIF

Podstawową operacją grafu jest **motylek**: dwie liczby A i B odległe o połowę bloku są zastępowane przez

$$A' = A + B, \qquad B' = (A - B) \cdot W_N^t.$$



Rysunek 5: Motylek DIF. Nazwa pochodzi od kształtu skrzyżowanych linii.

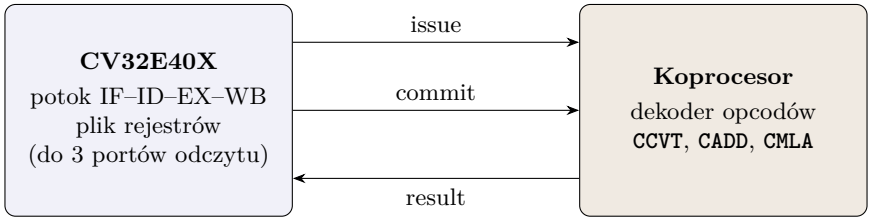
2 Platforma sprzętowa: rdzeń CV32E40X z koprocesorem

2.1 Rdzeń CV32E40X i interfejs rozszerzeń

Obliczenia wykonywane są na 32-bitowym rdzeniu RISC-V **CV32E40X** (OpenHW Group). Rdzeń ma 4-stopniowy potok z kolejnym kończeniem instrukcji: pobranie (IF), dekodowanie z odczytem rejestrów (ID), wykonanie (EX) i zapis wyniku (WB). Rdzeń nie ma jednostki zmiennoprzecinkowej ani instrukcji arytmetyki zespolonej.

Rdzeń udostępnia opcjonalny interfejs rozszerzeń **CORE-V-XIF** (*eXtension Interface*), włączany parametrem `X_EXT`. Pozwala on dodać własne instrukcje *bez zmiany kodu RTL rdzenia*: rozszerzenie jest osobnym modułem (koprocesorem) łączonym z rdzeniem na poziomie systemu. Do koprocesora można skierować wszystkie opcody, których rdzeń sam nie obsługuje. Przez interfejs nie można realizować skoków ani rozgałęzień, natomiast można realizować instrukcje arytmetyczne, odwołania do pamięci oraz własne rejestry CSR. Nasze instrukcje należą do pierwszej grupy.

Rdzeń nie zawiera żadnej gotowej obsługi liczb zespolonych: koprocesor z instrukcjami `CCVT`, `CADD` i `CMLA` jest modułem dopisywanym w ramach projektu i podłączanym do interfejsu XIF.



Rysunek 6: Połączenie rdzenia z koprocesorem. Strzałki oznaczają kierunek przepływu danych w interfejsach XIF używanych w projekcie.

2.2 Przebieg wykonania instrukcji

Instrukcja rozszerzenia wykonywana jest w następujących krokach:

1. Rdzeń dekoduje instrukcję w stopniu ID. Ponieważ nie jest to instrukcja rdzenia, przesyła ją do koprocesora razem z wartościami rejestrów wskazanych w polach `rs1` (bity 19:15), `rs2` (24:20) i `rs3` (31:27) instrukcji oraz z identyfikatorem instrukcji (interfejs `issue`). Bity `rs_valid` informują, które argumenty są już ważne.

2. Koprocesor rozpoznaje instrukcję po opcodzie. Przyjmuje ją tylko wtedy, gdy potrafi ją zdekodować i ważne są argumenty, których potrzebuje. W odpowiedzi zgłasza też, że wynik zapisze do rejestru `rd`.

3. Dla każdej przesłanej instrukcji rdzeń wystawia dokładnie jeden raz informację, czy instrukcja zostaje wykonana, czy anulowana, np. po skoku lub wyjątku (interfejs `commit`). Koprocesor nie może rozpocząć wysyłania wyniku, dopóki nie zobaczy zatwierdzenia instrukcji.

4. Koprocesor oblicza wynik i odsyła go interfejsem `result`. Rdzeń zapisuje go do rejestru `rd` w stopniu WB.

Każda przyjęta, zatwierdzona i nieanulowana instrukcja dostaje dokładnie jedną transakcję `result`. Wyniki mogą wracać w innej kolejności niż instrukcje były wydane (instrukcje rozróżnia identyfikator). Wynik wraca przez uzgadnianie `valid`/`ready`, więc koprocesor może liczyć dowolną liczbę cykli. Dotyczy to w szczególności `CMLA`, które zawiera cztery mnożenia. Koprocesor nie przechowuje własnego stanu (wszystkie dane są w rejestrach rdzenia), więc anulowanie instrukcji sprowadza się do tego, że jej wynik nie jest zapisywany.

Dokumentacja rdzenia zaleca, aby sygnały XIF bez osobno podanego budżetu czasowego dzieliły okres zegara w proporcji 20% procesor, 20% połączenie (interkonekt), 60% koprocesor. Jest to ograniczenie dla logiki kombinacyjnej koprocesora, w tym dla mnożników w `CMLA`.

2.3 Parametry rdzenia

Parametry rdzenia związane z interfejsem rozszerzeń zestawiono w tab. 3. Te same wartości muszą być ustawione w rdzeniu, w instancji interfejsu `cv32e40x_if_xif` oraz w koprocesorze.

parametr	wartość	uzasadnienie
X_EXT	1	włączenie interfejsu rozszerzeń (domyślnie 0)
X_NUM_RS	3	**CMLA** ma trzy argumenty źródłowe (rs1, rs2, rs3); domyślnie 2, dopuszczalny zakres 2–3
X_RFR_WIDTH	32	jedna liczba zespolona zajmuje jeden rejestr 32-bitowy
X_RFW_WIDTH	32	jw., dla zapisu wyniku
X_MEM_WIDTH	32	wartość domyślna; instrukcje nie odwołują się do pamięci
X_ID_WIDTH	4	wartość domyślna (identyfikator instrukcji w locie)
X_MISA	32'h0	koprocesor nie ogłasza standardowych rozszerzeń w rejestrze misa
X_ECS_XS	2'b0	koprocesor nie ma własnego stanu kontekstu, więc mstatus.XS = 0

Tabela 3: Parametry rdzenia CV32E40X dla interfejsu rozszerzeń.

Przy X_EXT = 1 granice regionów w konfiguracji PMA muszą być wyrównane do X_MEM_WIDTH bitów. Wartość X_NUM_RS = 3 oznacza, że rdzeń ma trzy porty odczytu pliku rejestrów (odczyt odbywa się w stopniu ID); jest to koszt powierzchni, ale potrzebny tylko dla **CMLA**.

2.4 Używane i nieużywane interfejsy XIF

Interfejs XIF składa się z sześciu części. Projekt wykorzystuje trzy z nich (tab. 4). Pozostałe muszą zostać zaślepione na poziomie integracji.

interfejs	użycie	rola
issue	tak	przekazanie instrukcji i rejestrów źródłowych do koprocesora; odpowiedź koprocesora (przyjęcie, odczyt rejestrów, zapis wyniku)
commit	tak	potwierdzenie lub anulowanie instrukcji
result	tak	wynik do zapisu w rejestrze rd
compressed	nie	instrukcje skompresowane nie są obsługiwane
mem	nie	instrukcje nie odwołują się do pamięci
mem_result	nie	jw.

Tabela 4: Interfejsy XIF w projekcie.

Zaślepienie polega na stałym odrzuceniu zapytań (dla compressed: gotowość 1 i brak przyjęcia instrukcji) oraz niewystawianiu żądań pamięci (mem_valid = 0).

2.5 Kodowanie instrukcji

Instrukcje kodowane są w opcodach zarezerwowanych w RISC-V na rozszerzenia własne: custom-0 (0x0B) i custom-1 (0x2B). Rdzeń dekoduje tylko opcody wymienione w typie opcode_e w pakiecie cv32e40x_pkg.sv: SYSTEM (0x73), FENCE (0x0F), OP (0x33), OP-IMM (0x13), STORE (0x23), LOAD (0x03), BRANCH (0x63), JALR (0x67), JAL (0x6F), AUIPC (0x17), LUI (0x37) i AMO (0x2F). Wybrane opcody 0x0B i 0x2B się tam nie znajdują, więc rdzeń uzna je za nieznane i zaoferuje koprocesorowi.

CCVT i **CADD** mają dwa argumenty źródłowe, więc używają formatu R. **CMLA** ma trzy argumenty źródłowe i używa formatu R4, w którym pole rs3 zajmuje bity, które w formacie R zajmuje funct7.

Z tego powodu **CMLA** ma inny opcode niż pozostałe instrukcje (rys. 7).

Format R (CCVT, CADD)					
31:25	24:20	19:15	14:12	11:7	6:0
funct7	rs2	rs1	funct3	rd	opcode

Format R4 (CMLA)						
31:27	26:25	24:20	19:15	14:12	11:7	6:0
rs3	funct2	rs2	rs1	funct3	rd	opcode

Rysunek 7: Formaty instrukcji. Numery to pozycje bitów w 32-bitowym słowie.

Parametry instrukcji (k, h, n, s, c) zapisane są w polach `funct` (tab. 5). Kombinacje niewymienione w tabeli, w tym pola oznaczone jako zarezerwowane różne od zera, koprocesor odrzuca (odpowiedź "nie przyjmuję"). Instrukcja, której nie przyjął ani rdzeń, ani koprocesor, kończy się wyjątkiem nielegalnej instrukcji zgłaszanym przy jej zatwierdzaniu.

instrukcja	format	opcode	`funct3`	parametry
CADD	R	0x0B (0001011)	000	`funct7[1:0]` $= k$, `funct7[3]` $= h$, `funct7[6:4]` i `[2]` $= 0$
CCVT	R	0x0B (0001011)	001	`funct7[1:0]` $= n$, `funct7[2]` $= s$, `funct7[3]` $= h$, `funct7[6:4]` $= 0$
CMLA	R4	0x2B (0101011)	$\{0, c, h\}$	`funct2` $= k$, `funct3[1]` $= c$, `funct3[0]` $= h$, `funct3[2]` $= 0$

Tabela 5: Kodowanie parametrów instrukcji rozszerzenia.

Przykładowe słowa instrukcji dla rejestrów $rd = x5$, $rs1 = x6$, $rs2 = x7$ (oraz $rs3 = x8$ dla **CMLA**):

- **CADD** z $k = 2$, $h = 1$ (czyli $(rs1 - rs2)/2$): 0x1473028B,
- **CCVT** z $n = 1$ (12 bitów), $s = 0$, $h = 0$: 0x0273128B,
- **CMLA** z $k = 0$, $c = 0$, $h = 1$: 0x407312AB.

2.6 Odpowiedź koprocesora na zapytanie issue

Odpowiedź koprocesora zawiera pola z tab. 6 (nazwy zgodne ze specyfikacją CORE-V-XIF, rewizja 458c8a73). Rdzeń nie dostaje od koprocesora listy rejestrów do odczytu: zawsze podaje argumenty z pól `rs1`, `rs2` i `rs3` instrukcji, a to koprocesor przyjmuje instrukcję dopiero wtedy, gdy ważne są argumenty, których potrzebuje. Dla **CCVT** i **CADD** są to rs1 i rs2, dla **CMLA** także rs3. Każdy argument i wynik jest liczbą zespoloną Q15 spakowaną w jednym rejestrze 32-bitowym (rys. 9).

pole odpowiedzi	wartość	znaczenie
`accept`	1	instrukcja przyjęta (dla niezakodowanych: 0)
`writeback`	1	koprocesor zapisze wynik do `rd`
`dualwrite`	0	wynik zajmuje jeden rejestr
`dualread`	0	argumenty zajmują po jednym rejestrze
`loadstore`	0	instrukcja nie odwołuje się do pamięci
`ecswrite`	0	koprocesor nie zapisuje `mstatus.XS/FS/VS`
`exc`	0	instrukcja nie zgłasza wyjątku

Tabela 6: Odpowiedź koprocesora na zapytanie `issue` dla instrukcji rozszerzenia.

W transakcji `result` koprocesor podaje identyfikator instrukcji, 32-bitowy wynik (`data`), numer rejestru `rd` i zezwolenie na zapis (`we` = 1). Pozostałe pola pakietu (`exc`, `exccode`, `ecswe`, `ecsdata`, `err`, `dbg`) mają wartość 0. Pakiet `commit` zawiera tylko identyfikator instrukcji (`id`) i flagę `commit_kill`.

2.7 Wywołanie instrukcji z programu

Kompilator nie zna nowych instrukcji, więc w kodzie C wywołuje się je makrami z wstawką asemblerową i dyrektywą `.insn` asemblera GNU (poniżej szkic, parametry `k`, `h` muszą być stałymi):

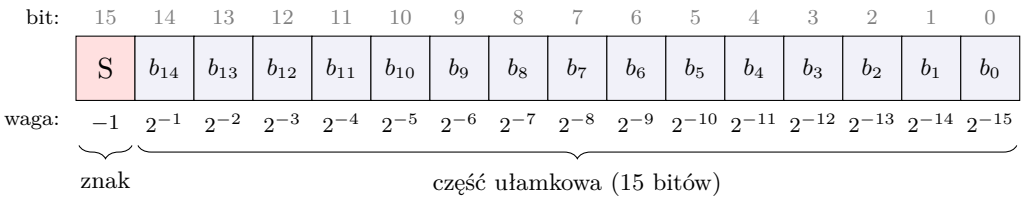
```
#define CADD(k, h, a, b) ({ uint32_t r;                          \
    asm volatile(".insn r 0x0B, 0, %3, %0, %1, %2"               \
                 : "=r"(r) : "r"(a), "r"(b),                     \
                   "i"(((h) << 3) | (k)));                       \
    r; })

#define CMLA(k, c, h, a, b, acc) ({ uint32_t r;                  \
    asm volatile(".insn r4 0x2B, %4, %5, %0, %1, %2, %3"         \
                 : "=r"(r) : "r"(a), "r"(b), "r"(acc),           \
                   "i"(((c) << 1) | (h)), "i"(k));               \
    r; })
```

3 Reprezentacja danych: format Q15

3.1 Liczby stałoprzecinkowe Q15

Rdzeń nie posiada jednostki zmiennoprzecinkowej, dlatego liczby rzeczywiste zapisywane są w formacie stałoprzecinkowym **Q15** (`ap_fixed<16,1>`). Liczba zajmuje 16 bitów w kodzie U2: najstarszy bit (15) jest bitem znaku o wadze -1, a pozostałe 15 bitów tworzy część ułamkową o wagach od 2^{-1} do 2^{-15} (rys. 8). Wartość liczby jest sumą wag bitów ustawionych na 1.



Rysunek 8: Układ bitów liczby Q15.

wartość	bity (15 . . . 0)	hex
-1	1000 0000 0000 0000	0x8000
$-0{,}5$	1100 0000 0000 0000	0xC000
-2^{-15}	1111 1111 1111 1111	0xFFFF
0	0000 0000 0000 0000	0x0000
$2^{-15} \approx 0{,}00003$	0000 0000 0000 0001	0x0001
$0{,}5$	0100 0000 0000 0000	0x4000
$\approx 0{,}7071$	0101 1010 1000 0010	0x5A82
$1 - 2^{-15} \approx 0{,}99997$	0111 1111 1111 1111	0x7FFF

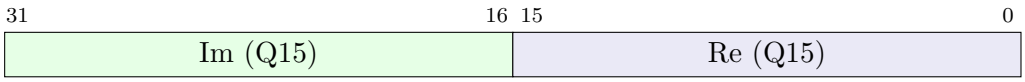
Tabela 7: Przykładowe wartości w formacie Q15. Przykładowo `0xC000` ma ustawione bity 15 i 14, więc jego wartość to $-1 + 0{,}5 = -0{,}5$.

Zakres formatu wynosi $[-1;\ 1 - 2^{-15}]$. Wartość $+1$ nie jest reprezentowalna, co ma znaczenie dla czynnika obrotu $W_N^0 = 1$: mnożenie przez niego należy pominąć, a nie wykonywać przez **`CMLA`**.

W kodzie HLS zastosowano tryb `AP_SAT` (nasycenie – wynik spoza zakresu jest ograniczany do -1 lub $1 - 2^{-15}$ zamiast przepełnienia) oraz `AP_RND` (zaokrąglanie zamiast obcinania).

3.2 Pakowanie liczby zespolonej w rejestr 32-bitowy

Jedna liczba zespolona zajmuje jeden rejestr RISC-V (RV32):



Rysunek 9: Liczba zespolona w rejestrze: bity [15:0] – część rzeczywista, bity [31:16] – część urojona.

Dzięki temu jedna instrukcja przetwarza jednocześnie obie składowe liczby zespolonej.

3.3 Skalowanie – flaga h

Suma dwóch liczb z zakresu $[-1, 1)$ może należeć do zakresu $[-2, 2)$, co powoduje przepełnienie. W FFT każdy etap może podwoić wartości, a po $\log_2 N$ etapach $|X[k]|$ może być N razy większe od wartości wejściowych. Instrukcje posiadają więc flagę h (*half*): dla $h = 1$ wynik jest dzielony przez 2. Dzielenie wykonywane jest na poszerzonym formacie pośrednim (`Q2_16`, `Q4_31`), dzięki czemu najstarszy bit nie jest tracony.

Przy $h = 1$ we wszystkich $\log_2 N$ etapach wynik końcowy wynosi

$$\frac{X[k]}{N},$$

i zawsze mieści się w zakresie Q15, ponieważ $|X[k]|/N \leq \max |x[n]|$.

4 Rozszerzenie RISC-V: instrukcje zespolone

Rozszerzenie obejmuje trzy instrukcje: **`CCVT`**, **`CADD`** i **`CMLA`**. Są one realizowane przez koprocesor podłączony do interfejsu XIF rdzenia CV32E40X; kodowanie instrukcji podaje tab. 5, a parametry rdzenia tab. 3. W opisie j^k oznacza obrót argumentu o $k \cdot 90°$ (tab. 8):

k	j^k	obrót	wynik dla $z = a + jb$
0	1	0°	$a + jb$
1	j	+90°	$-b + ja$
2	-1	180°	$-a - jb$
3	$-j$	−90°	$b - ja$

Tabela 8: Parametr k – obrót o wielokrotność 90°, realizowany przez zamianę i negację składowych.

4.1 `CCVT` – konwersja próbek z ADC

$$\texttt{CCVT}(n, s, h, rs1, rs2) \;\rightarrow\; rd = \big(\mathrm{Re} = \mathrm{conv}(rs1),\ \mathrm{Im} = \mathrm{conv}(rs2)\big)$$

parametr	szerokość	znaczenie
n	2 bity	rozdzielczość przetwornika: `0` – 10 bitów, `1` – 12 bitów, `2` – 14 bitów, `3` – 16 bitów
s	1 bit	format próbki: `0` – bez znaku (*offset binary*), `1` – ze znakiem (U2)
h	1 bit	`1` – wynik dzielony przez 2
$rs1$	16 bitów	próbka zapisywana jako część rzeczywista (młodsze 16 bitów rejestru)
$rs2$	16 bitów	próbka zapisywana jako część urojona (młodsze 16 bitów rejestru)
rd	32 bity	liczba zespolona Q15

Tabela 9: Parametry instrukcji `CCVT`.

Instrukcja zamienia dwie próbki z przetwornika analogowo-cyfrowego na jedną liczbę zespoloną Q15 w następujących krokach:

1. **wyrównanie do 16 bitów** – przesunięcie w lewo zależne od n: 10 bitów → `<<6`, 12 bitów → `<<4`, 14 bitów → `<<2`, 16 bitów → bez zmian;
2. **zmiana kodu** – dla $s = 0$ negowany jest najstarszy bit, co zamienia kod z przesunięciem (środek zakresu = `0x8000`) na kod U2 (środek = 0);
3. **skalowanie** – dla $h = 1$ arytmetyczne przesunięcie w prawo o 1 bit;
4. **pakowanie** – $rs1$ → bity [15:0] (Re), $rs2$ → bity [31:16] (Im).

próbka 12-bit	po `<<4`	po negacji MSB	wartość Q15
`0x000` (min)	`0x0000`	`0x8000`	-1
`0x800` (środek)	`0x8000`	`0x0000`	0
`0xFFF` (max)	`0xFFF0`	`0x7FF0`	$\approx 0{,}9995$

Tabela 10: Przykład działania `CCVT` dla $n =$ `BIT12`, $s = 0$, $h = 0$.

Dla sygnału rzeczywistego z jednego kanału jako $rs2$ należy podać wartość odpowiadającą zeru po konwersji (`0x800` dla próbek 12-bitowych przy $s = 0$). Podanie wartości `0` dałoby część urojoną równą -1. Alternatywnie $rs1$ i $rs2$ mogą zawierać próbki dwóch kanałów (np. sygnały I/Q).

4.2 `CADD` – dodawanie z obrotem

$$\texttt{CADD}(k, h, rs1, rs2) = \frac{rs1 + j^k \cdot rs2}{2^h}$$

parametr	szerokość	znaczenie
k	2 bity	obrót $rs2$ o $k \cdot 90°$ przed dodaniem (tab. 8): `0` – dodawanie, `2` – odejmowanie, `1/3` – dodanie $rs2$ pomnożonego przez j / $-j$
h	1 bit	`1` – wynik dzielony przez 2
$rs1$, $rs2$	32 bity	argumenty zespolone Q15
rd	32 bity	wynik zespolony Q15

Tabela 11: Parametry instrukcji `CADD`.

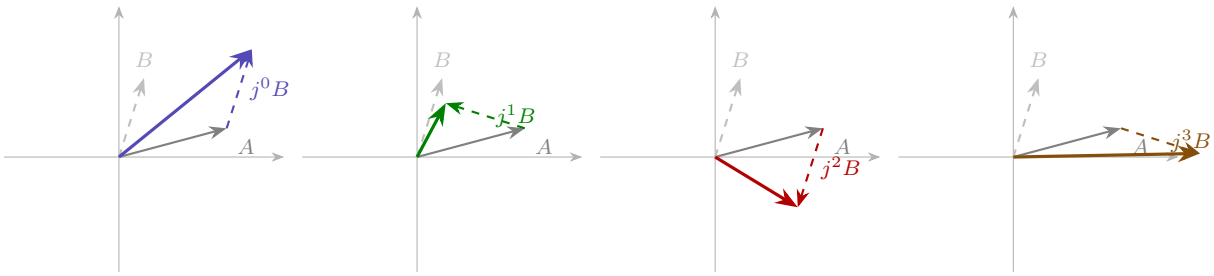
W rozpisaniu na składowe, dla $rs1 = a + jb$, $rs2 = c + jd$ i $h = 0$:

$$\begin{aligned} k = 0: &\quad \mathrm{Re} = a + c, &\quad \mathrm{Im} = b + d, \\ k = 1: &\quad \mathrm{Re} = a - d, &\quad \mathrm{Im} = b + c, \\ k = 2: &\quad \mathrm{Re} = a - c, &\quad \mathrm{Im} = b - d, \\ k = 3: &\quad \mathrm{Re} = a + d, &\quad \mathrm{Im} = b - c. \end{aligned}$$

Dla $rs1 = 0$ instrukcja realizuje samo mnożenie przez j^k, np.

$$\texttt{CADD}(3, 0, 0, rs2) = -j \cdot rs2,$$

czyli mnożenie przez $-j = W_N^{N/4}$ bez użycia mnożarki.



$k = 0$: $A + B$ $k = 1$: $A + jB$ $k = 2$: $A - B$ $k = 3$: $A - jB$

Rysunek 10: Działanie `CADD`: do A dodawany jest wektor B obrócony o $k \cdot 90°$.

4.3 `CMLA` – zespolone mnożenie z akumulacją

$$\texttt{CMLA}(k, c, h, rs1, rs2, rs3) = \frac{rs3 + j^k \cdot (rs1 \cdot \widetilde{rs2})}{2^h}, \qquad \widetilde{rs2} = \begin{cases} rs2, & c = 0 \\ \overline{rs2}, & c = 1 \end{cases}$$

parametr	szerokość	znaczenie
k	2 bity	obrót iloczynu o $k \cdot 90°$ przed dodaniem do $rs3$ (tab. 8); `2` daje odejmowanie iloczynu
c	1 bit	`1` – użycie sprzężenia $\overline{rs2}$ zamiast $rs2$
h	1 bit	`1` – wynik dzielony przez 2
$rs1$, $rs2$	32 bity	czynniki iloczynu (zespolone Q15)
$rs3$	32 bity	składnik dodawany do iloczynu (akumulator)
rd	32 bity	wynik zespolony Q15

Tabela 12: Parametry instrukcji **`CMLA`**.

Instrukcja ma trzy rejestry źródłowe, podobnie jak `fmadd` w RISC-V. Dla $rs1 = a + jb$, $rs2 = c' + jd$ iloczyn wynosi:

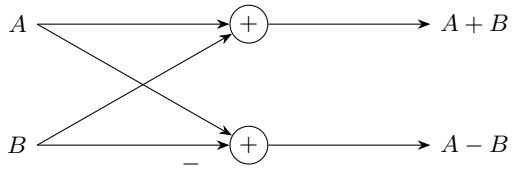
$$\begin{aligned} c = 0: &\quad rs1 \cdot rs2 = (ac' - bd) + j(ad + bc'), \\ c = 1: &\quad rs1 \cdot \overline{rs2} = (ac' + bd) + j(bc' - ad). \end{aligned}$$

Iloczyny cząstkowe przechowywane są w formacie `Q3_30`, a dodanie $rs3$ i ewentualne dzielenie przez 2 odbywa się przed zaokrągleniem do Q15. Błąd zaokrąglenia powstaje więc tylko raz.

4.4 Realizacja motylków za pomocą instrukcji

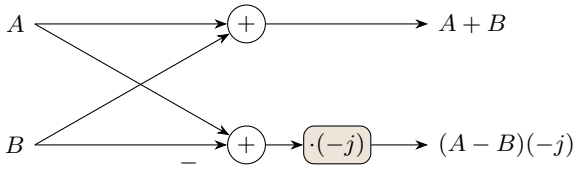
Poniżej zestawiono warianty motylka spotykane w algorytmach FFT wraz z odpowiadającymi im sekwencjami instrukcji. Każdy motylek wykonywany jest przez kilka kolejnych instrukcji, bez pętli i rozgałęzień.

Wariant 1: DIF, $W = 1$. Motylek bez czynnika obrotu, np. w ostatnim etapie FFT.



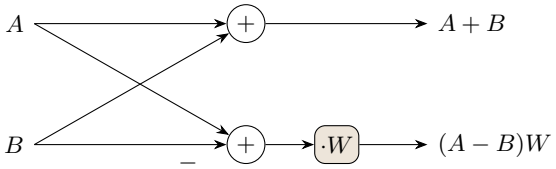

```
A1 = CADD(0, h, A, B);      // (A + B) / 2^h
B1 = CADD(2, h, A, B);      // (A - B) / 2^h
```

Wariant 2: DIF, $W = -j$. Mnożenie przez $W_N^{N/4} = -j$ wykonuje **`CADD`** z $k = 3$.



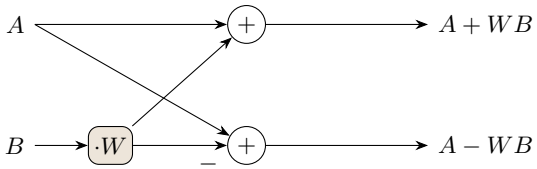

```
A1 = CADD(0, h, A, B);      // (A + B) / 2^h
T  = CADD(2, h, A, B);      // (A - B) / 2^h
B1 = CADD(3, 0, 0, T);      // -j * T
```

Wariant 3: DIF, dowolne W. Mnożenie po odejmowaniu wykonuje **CMLA** z $rs3 = 0$. Skalowanie zostało już wykonane w **CADD**, a $|W| = 1$, dlatego **CMLA** używa $h = 0$.



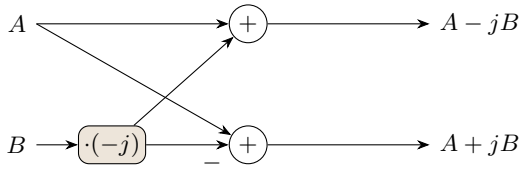

```
A1 = CADD(0, h, A, B);              // (A + B) / 2^h
T  = CADD(2, h, A, B);              // (A - B) / 2^h
B1 = CMLA(0, 0, 0, T, W, 0);        // T * W
```

Wariant 4: mnożenie przed sumowaniem, dowolne W. Motylek postaci $A \pm WB$ (stosowany m.in. w decymacji w czasie). Mnożenie i dodawanie łączą się w jedną instrukcję **CMLA**.



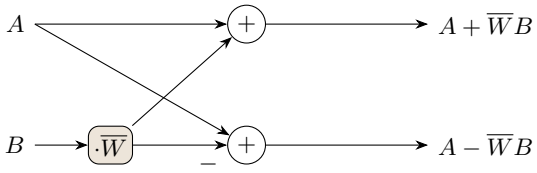

```
A1 = CMLA(0, 0, h, B, W, A);        // (A + W*B) / 2^h
B1 = CMLA(2, 0, h, B, W, A);        // (A - W*B) / 2^h
```

Wariant 5: mnożenie przed sumowaniem, $W = -j$. Ponieważ $A + (-j)B = A + j^3B$ oraz $A - (-j)B = A + j^1B$, motylek realizują dwie instrukcje **CADD**, bez mnożarki.




```
A1 = CADD(3, h, A, B);      // (A - jB) / 2^h
B1 = CADD(1, h, A, B);      // (A + jB) / 2^h
```

Wariant 6: mnożenie przez sprzężenie (transformata odwrotna). Odwrotna FFT używa czynników $W_N^{-t} = \overline{W_N^t}$. Flaga $c = 1$ pozwala skorzystać z tej samej tablicy czynników obrotu.




```
A1 = CMLA(0, 1, h, B, W, A);        // (A + conj(W)*B) / 2^h
B1 = CMLA(2, 1, h, B, W, A);        // (A - conj(W)*B) / 2^h
```

wariant	operacja	instrukcje	w tym **CMLA**
1	$A+B,\ A-B$	2	0
2	$A+B,\ (A-B)(-j)$	3	0
3	$A+B,\ (A-B)W$	3	1
4	$A+WB,\ A-WB$	2	2
5	$A-jB,\ A+jB$	2	0
6	$A+\overline{W}B,\ A-\overline{W}B$	2	2

Tabela 13: Koszt poszczególnych wariantów motylka.

4.5 Przykład: FFT 4-punktowa bez pętli

Dla $N = 4$ jedynymi czynnikami obrotu są $W_4^0 = 1$ i $W_4^1 = -j$, więc całą transformatę wykonuje się bez mnożarki. Etap 1 to warianty 1 i 2, etap 2 – dwukrotnie wariant 1:

```
// etap 1: pary (x0, x2), (x1, x3)
b0 = CADD(0, h, x0, x2);
b2 = CADD(2, h, x0, x2);
b1 = CADD(0, h, x1, x3);
t  = CADD(2, h, x1, x3);
b3 = CADD(3, 0, 0,  t);     // * W_4^1 = -j
// etap 2: pary (b0, b1), (b2, b3)
X0 = CADD(0, h, b0, b1);
X2 = CADD(2, h, b0, b1);
X1 = CADD(0, h, b2, b3);
X3 = CADD(2, h, b2, b3);
```

Mnożenie przez $-j$ można połączyć z etapem 2 (wariant 5), co skraca sekwencję do 8 instrukcji:

```
b0 = CADD(0, h, x0, x2);
b2 = CADD(2, h, x0, x2);
b1 = CADD(0, h, x1, x3);
t  = CADD(2, h, x1, x3);
X0 = CADD(0, h, b0, b1);
X2 = CADD(2, h, b0, b1);
X1 = CADD(3, h, b2, t);     // b2 + (-j)t
X3 = CADD(1, h, b2, t);     // b2 - (-j)t
```

Przy $h = 1$ wyniki wynoszą $X[k]/4$.